

Split Creator - Component Design

Component Design

The user interacts top-down, from `c-splitConfigurator` down to `c-splitDetails`, while each child reports its own state and validity back up, the same pattern used elsewhere in this design (see the Spot Durations, Dayparts and Bursts component). Nothing here is saved directly by a child component: `c-splitDetails` builds the payload, and the page-level save (Section: Save Behaviour) writes it as Linear Parameter records.



c-splitConfigurator

Renders the four goal toggles (Fixed Budget, Fixed Price, Fixed Audience, Highest Reach) described in Function of the Component. Selecting Fixed Price reveals the CPM/CPT choice; selecting Fixed Audience reveals the 000s/TARPs choice. Below the goals, a row of tiles for the

six split types (Budget, TARPs, 000s, Spots, CPM, CPT) lets the user add further splits beyond whatever the goals themselves require.

- Selecting a goal that requires a fixed split (Budget, Price, or Audience) creates the corresponding `c-split` instance automatically, with a count badge on its tile, and locks that split against removal while the toggle stays on. This is a display-only lock enforced by `c-splitConfigurator`: it does not itself write anything to a Linear Parameter record.
- Deselecting the goal toggle unlocks the split for removal but does not delete it automatically. This is deliberately different from the Bursts/Custom Dayparts deletion rule in Navigation Behaviour, which is a data-integrity rule (stale references to changed brief data), not a goal-management rule.
- Owns no split values itself. It holds only the goal/toggle state and the current list of splits (one `c-split` instance per goal-driven or manually added split type), and aggregates every `c-split` child's reported validity to gate the page-level save.

c-split

One instance per split. Represents the split's parent Linear Parameter record (Data Model, Section 6) and is the component that carries `TypePick__c`, `MetricPick__c`, `SplitColumnDriverPick__c`, `SplitChildGroupPick__c`, `SplitParentGroupPick__c` and `SplitGrandarentGroupPick__c` as its own configuration state.

- Renders the split's header (its type and, where relevant, its CPM/CPT or 000s/TARPs qualifier inherited from `c-splitConfigurator`) and a gear-icon settings panel.
- The settings panel holds up to three numbered Group By checkboxes (Market, Channel, Daypart, Duration, Day of the Week, or a custom time range for Fixed-only campaigns), numbered by click order rather than a separate reorder control, and one Column Driver radio group (No column split / Split by Week / Split by Burst). An option that isn't available for the current campaign, for example Split by Burst with no bursts configured, renders disabled with an explanatory message rather than being hidden, consistent with the disabled-not-hidden pattern used for other unavailable options in this design.
- For a Spots-type split only, also renders the %/# toggle mentioned in Function of the Component.
- Hosts one `c-splitDetails` (the grid) and one `c-splitActions` (the toolbar) as children, and passes its Group By selection, Column Driver, and (for Spots) %/# state down to `c-splitDetails` whenever it changes, so the grid can reshape its rows and columns.

c-splitDetails

The grid itself, and the component that actually builds the JSON. Rows come from the Group By dimensions selected on `c-split`, nested up to three levels deep; columns come from the Column Driver (none, one per week, or one per burst).

- Holds every entered value, in the unit assigned to the split, plus the derived totals: per row, per level, per column, and for the split overall.
- Drives the status banner against the split's target: a fixed value for a fixed-unit split, 100% for a percentage split. When a fixed goal's total hasn't been entered, shows the inline "what's missing" message anchored to the row it concerns (the CPM/dollar-rate example already given in Function of the Component) rather than a generic validation error.
- Distinguishes a grouped row from an ungrouped one visually, so the user can tell at a glance which rows represent a combined value.
- Builds the actual save payload: when the split has no column driver, everything it holds becomes the parent record's `SplitJSON__c` (row totals only). When a column driver is set, it builds the parent's totals JSON and one child JSON per week or burst, matching the shape in JSON Design. It reports its own validity (every required row/level populated, no unreconciled fixed total) up to `c-split`.

c-splitActions

A stateless action bar. It reads whatever `c-splitDetails` currently has selected and fires: group, ungroup, duplicate, reorder, delete; an even split applied across a selection's children and across columns; and copying a set of values to every equivalent row at another level, or across all weeks.

- Holds no data of its own, and never writes to a Linear Parameter record directly. Every action it fires is carried out by `c-splitDetails` against its own row and column state, the same relationship `c-splitConfigurator` has to `c-split`: the action bar orchestrates, the grid owns the data.

SplitSaveService (Apex)

Not an LWC. This is the class that actually turns `c-splitDetails`' built payload into Linear Parameter DML, and it's missing from the document as it stands: the Data Model section defines the fields, but nothing currently names what writes them. Named `SplitSaveService` per this project's `{Feature}Service` convention, called imperatively (`@AuraEnabled`) from `c-splitConfigurator` at the two save points Save Behaviour already defines: Save & Close, and Run Optimiser.

- Takes every `c-split` instance's payload from `c-splitDetails` (one parent, plus zero or more children where a column driver is set) and upserts them in a single transaction: one

parent Linear Parameter record per split, one child record per week or burst column. Uses `allOrNone = false` and logs any partial failure, rather than letting one bad split silently fail the whole save.

- Reads existing records on open via the shared `LinearParameterSelector`, the same selector already used for Bursts and Dayparts, so `c-splitConfigurator` can restore a previously saved configuration. No Split-specific selector is needed; it's the same object.
- On Run Optimiser, after a successful save, a further step sends the saved records' IDs and External Identifiers to the OMS to initiate Day8 optimisation, per Save Behaviour. Worth deciding before build whether that's a second method on `SplitSaveService` or a separate class (for example `SplitOptimiserRequestService`, calling out via a Named Credential per this project's integration standard). Either way, it shouldn't live inside `SplitSaveService`'s own DML logic: persistence and the OMS callout are different responsibilities and should stay separate classes.
- Runs `with sharing`, enforces FLS on every record before DML, and validates each payload it receives rather than trusting client-supplied JSON as-is.